

DocExtract — build timeline

The order to write the 31 `src` modules, sequenced so that **when you write a module, everything it depends on already exists**. This isn't arbitrary — it follows the real dependency graph (traced from the imports). You physically cannot write `extractor` before `routing` and `schema` exist, because it calls them; this plan respects that throughout.

Two things are layered on top of the order:

- **Effort** in points, where **1 point = half a day** of focused work (your scale). These price *authoring + the thinking + the false starts*, not just typing — see the note at the end on why that matters.
- A **week-by-week schedule** assuming ~5 productive points/week for one engineer (about 2.5 focused days of build after meetings/reviews/life).

The shape of the build

Modules fall into **7 dependency waves**. You build wave by wave; within a wave, modules are independent of each other so the order is flexible.

```
Wave 0 foundations (depend on nothing internal)
Wave 1 first consumers (need only foundations)
Wave 2 composers (need wave-1 pieces)
Wave 3 the heavy hitters (extractor, monitoring)
Wave 4 the orchestrator (production — wires it all)
Wave 5 the entry point (run_ingestion_job)
Wave 6 the job wrapper (workflow_tasks)
```

The shape is a diamond: a wide base of small independent modules, narrowing to the single orchestrator that depends on almost everything, then a thin tail of entry points. **Most of the risk is in Wave 3-4**, not the beginning.

Wave 0 — Foundations (write these first)

Nothing internal depends on these; everything depends on *them*. Get `schema` and `config` right and the rest of the build is smoother — get them wrong and you'll be refactoring for weeks.

Order	Module	Points	Notes
1	schema	3	The data contracts. The most consequential file — every module speaks it. Worth over-investing here.
2	config	2	The control panel. Large but mostly declarative.
3	dictionary	1	Term→canonical mapping. Small, pure.
4	audit_log	1	Structured event logging.
5	gates	1	Governance checkpoints. Standalone.
6	cost_tracker	1	Cost projection. Standalone.
7	ai_functions_config	1	Native-track config. Declarative.

Wave 0 subtotal: ~10 points (~2 weeks)

Milestone: the vocabulary and settings exist. Nothing runs yet, but everything downstream now has a foundation to build on.

Wave 1 — First consumers (need only foundations)

These each depend on `schema` and/or `config` and nothing else. They're independent of each other, so order within the wave is free.

Order	Module	Points	Notes
8	llm_client	3	The switchboard to the models. Tool-schema + multimodal. Test against a real endpoint early.
9	routing	2	Content-aware tier decision.
10	metric_normaliser	2	Structural shape + compatibility.
11	storage	3	Medallion writes, partitioning, idempotency.
12	ai_search_store	3	Azure AI Search index + hybrid query. Real integration risk.
13	figure_preprocessor	2	Crop-and-zoom + page-image provider.
14	prompt_registry	2	Stored prompts as data.
15	eval_registry	2	Stored expected values as data. Mirrors prompt_registry.
16	supervisor_session	3	Memory + audit + feedback in one store.
17	reviewer	1	Applies a human decision.
18	validator	2	Reconcile + gate.
19	ai_parse_client	2	OCR fallback.
20	vector_sync	1	Native→search bridge.
21	native_pipeline	2	Declarative alternative track.

Wave 1 subtotal: ~30 points (~6 weeks)

This is the longest wave — it's the bulk of the independent building blocks. Milestone: all the pieces exist as testable units, even though nothing is wired end to end yet.

Wave 2 — Composers (need wave-1 pieces)

Order	Module	Points	Notes
22	prompts	1	Builds the extraction prompt; uses prompt_registry.
23	preprocessor	4	Big one. 5 formats + OCR fallback + bbox + footnote logic.
24	claim_quant_matcher	2	Retrieve-then-confirm linking.
25	llm_router	3	The librarian. Classify + SQL + narrative + reference-rewrite.
26	evaluator	2	Sampled scoring against independent truth.

Wave 2 subtotal: ~12 points (~2.5 weeks)

Milestone: the query side (llm_router) and the eval side (evaluator) work standalone; the parser handles every format.

Wave 3 — The heavy hitters

Order	Module	Points	Notes
27	extractor	4	The hardest single module — cascade, escalation, crop-zoom, provenance. Depends on 8 other modules; this is where the design is proven or found wanting.
28	monitoring	2	Aggregate dashboards; pulls from evaluator + supervisor_session.

Wave 3 subtotal: ~6 points (~1 week)

Milestone: extraction works on a single element end to end. The core value of the pipeline is now demonstrable.

Wave 4 — The orchestrator

Order	Module	Points	Notes
29	production	3	Wires nearly everything: preprocess → extract → reconcile → gate → store → index → eval. Depends on 9 modules. Low new-logic, high integration — this is where interface mismatches surface.

Wave 4 subtotal: ~3 points (but expect +integration debugging)

Milestone: a whole document flows through every stage. First true end-to-end run.
Budget extra time here for the interface reckoning — modules that seemed fine in isolation reveal mismatches when wired.

Apps — the human-facing UIs (write once their modules exist)

The three Streamlit apps aren't `(src)` modules, but they're real deliverables someone writes, and they sit at specific points in the dependency order because each imports from the package. They can be written as soon as their dependencies exist — you don't have to wait for the whole pipeline.

Order	App	Points	Writable after	Depends on	What it does
—	supervisor_app	2	Wave 2	llm_router, supervisor_session, ai_search_store	Front desk: ask questions, get grounded answers with memory + feedback
—	analyst_app	2	Wave 3	extractor, preprocessor	Inspect one document's extraction live, before it's trusted
—	monitoring_app	2	Wave 3	monitoring	Aggregate health dashboard across all runs

Apps subtotal: ~6 points (~1.5 weeks)

These are UI work — moderate effort each, mostly Streamlit layout plus wiring to already-built functions. `(supervisor_app)` can be built early (after Wave 2) which is useful: it lets you *demo the query side* long before the full pipeline is wired, which is good for stakeholder buy-in. `(analyst_app)` and `(monitoring_app)` need Wave 3 (extractor / monitoring) first.

Milestone: the three human touchpoints exist — a supervisor can ask questions, an analyst can inspect an extraction, and ops can watch system health.

Wave 5-6 — Entry points

Order	Module	Points	Notes
30	run_ingestion_job	1	Volume scan + filename→entity. Thin.
31	workflow_tasks	2	The three Databricks Job tasks (ingest/index/health).

Wave 5–6 subtotal: ~3 points (~1.5 weeks)

Milestone: the pipeline runs as a scheduled job. Deployable.

The timeline at a glance

Phase	Waves	Points	Approx. weeks	What exists at the end
Foundations	0	10	~2	The vocabulary + settings
Building blocks	1	30	~6	All units exist, testable
Composition	2	12	~2.5	Query + eval + parsing work
Core	3	6	~1	Extraction demonstrable
Wiring	4	3 (+debug)	~1.5	First end-to-end run
Apps	after 2–3	6	~1.5	The three human-facing UIs
Go-live	5–6	3	~1	Scheduled, deployable
Total (modules + apps)		~70	~15	The <code>src</code> package + UIs

Add tests (~15 pts), integration/debugging beyond the wiring estimate (~10 pts), and the up-front design work that precedes wave 0 (~8 pts), and the honest **from-scratch total lands around 103 points — roughly 5–5.5 months** for one engineer building it thoughtfully the first time.

Honest notes on these numbers

Why "modules only" (~64) is lower than the real build (~95+). Writing the modules is not the whole job. Before wave 0 there's design discovery — deciding there *should be* a two-axis trust model, a pre-call router, prompts-as-data. That thinking is invisible in the final code but it's real time, and it mostly happens by building the wrong version first. The module estimates price each file; they don't price the "I designed this wrong and rebuilt it" that a true first build always includes.

The estimates are front-loaded with uncertainty in the wrong place. These numbers look precise per module, but the real risk isn't spread evenly — it concentrates in `schema` (wave 0, because getting the contracts wrong ripples everywhere), `extractor` (wave 3, the hardest logic), and `production` (wave 4, where integration bites). If those three go

smoothly the whole thing tracks ahead of estimate; if any fights back, it slips. Watch those three.

This is one engineer's first-time build. A senior engineer who has built extraction pipelines before might do it in half — for them the design is recall, not discovery. Someone newer could take longer. And nobody builds fully cold: with references and libraries the real number is lower than 95. Treat this as the "genuinely thinking it through yourself" figure, and scale to your team's actual pace once a few modules are done and you know your real velocity.

The dependency order is the durable part. Effort estimates are soft and will shift with who's building. The *order* — foundations before consumers, the orchestrator near the end — is fixed by the imports and won't change. If you take one thing from this document, take the sequence, not the point totals.

The one-line version

Build the foundations (`schema`), (`config`) first, then the wide base of independent modules, then the composers, then the extractor, then wire it all together in (`production`), add the three apps once their modules exist, and finish with the entry points. ~15 weeks to write the modules + apps; ~5.5 months to build the whole thing from scratch, thoughtfully.